Listing 10.14 Configuring Cloud Stream bindings and RabbitMQ destinations

```
spring:
  cloud:
    function:
      definition: pack|label
    stream:
      bindings:                                  ❶
        packlabel-in-0:                          ❷
          destination: order-accepted            ❸
          group: ${spring.application.name}      ❹
        packlabel-out-0:                         ❺
          destination: order-dispatched          ❻
```

❶ Section for configuring destination bindings

❷ The input binding

❸ The actual name at the broker that the binder binds to (the exchange in RabbitMQ)

❹ The consumer group interested in the destination (same as the application name)

❺ The output binding

❻ The actual name at the broker that the binder binds to (the exchange in RabbitMQ)

The output binding (`packlabel-out-0`) will be mapped to an `order-dispatched` exchange in RabbitMQ. The input binding (`packlabel-in-0`) will be mapped to an `order-accepted` exchange and an `order-accepted.dispatcher-service` queue in RabbitMQ. If they don't exist already in RabbitMQ, the binder will create them. The queue-naming strategy (`<destination>.<group>`) includes a parameter called *consumer group*.

The idea of *consumer groups* has been borrowed from Kafka and is very useful. In a standard pub/sub model, all consumers receive a copy of the messages sent to the queues they're subscribed to. That is convenient when different applications need to process the messages. But in a cloud native context, where multiple instances of an application are running